

Cours de Programmation en C

Travaux Pratiques

— TP2 —

Exercice 1 – Tri par sélection

Dans cet exercice, on cherche à écrire un programme qui lit un tableau de réels dans un fichier texte, puis le trie et l'affiche.

1. Écrire un programme qui lit un fichier texte `"valeurs.txt"` contenant :

- le nombre de valeurs du fichier (noté n)
- Des valeurs réelles séparées par un espace

et les stocke dans un tableau T . On sait qu'il n'y a pas plus de 100 valeurs dans le fichier.
À la fin du programme, on n'oubliera pas de fermer le fichier.

2. En commentaires, écrire l'algorithme (sous forme de pseudo-code) de *tri par sélection* qui se base sur le principe suivant : on sélectionne tout d'abord l'élément le plus petit du tableau (on trouve son indice). Une fois cet indice p trouvé, on échange les éléments $T[0]$ et $T[p]$. Puis on recommence ces opérations pour le reste du tableau (c'est-à-dire les éléments compris entre les indices 1 et n). On recherche alors le plus petit élément de cette nouvelle suite de nombres et on échange avec $T[1]$. Et ainsi de suite jusqu'au moment où on a placé tous les éléments du tableau.
3. En déduire le programme qui trie le tableau T . Le tester avec le fichier trouvé dans le repertoire .
4. Terminer le programme en écrivant le tableau de valeurs triées dans le fichier `"valeurs-triees.txt"`.

Solution:**Algorithme 1 : Tri par sélection****Données :** T , le tableau à trier, n le nombre d'éléments de L **Sorties :** T la liste triée par ordre croissant

```

1 pour iter de 0 à  $n-1$  faire
2   imin  $\leftarrow$  iter
3   pour i de iter+1 à  $n-1$  faire
4     si  $T[i] < T[imin]$  alors
5       imin  $\leftarrow$  i
6   fin
7 fin
8 échanger  $T[iter]$  et  $T[imin]$ 
9 fin

```

```

/*=====
*
*      oO  Tri sélection  Oo
*
*=====
*
* File : tri-selection.c
* Date : 14 oct 12
* Author : Hilaire Thibault
*
*=====
*/

```

```

#include <stdio.h>
#include <stdlib.h>

```

```

#define TAILLE_MAX 100

```

```

int main()
{
    FILE* fichier;           /* descripteur de fichier */
    float T[TAILLE_MAX];     /* tableau à trier */
    float temp;
    int taille;               /* taille du tableau */
    int i, iter;
    int imin;                 /* indice de l'élément le plus petit */

    /* ouverture du fichier */
    fichier = fopen( "valeurs.txt", "r");
    if (fichier==NULL)
    {
        printf("Problème_d'ouverture_du_fichier\n");
        return EXIT_FAILURE;
    }

    /* lecture du fichier */
    fscanf( fichier, "%d", &taille);
    for(i=0;i<taille;i++)
        fscanf( fichier, "%f", &T[i]);

    /* fermeture du fichier */
    fclose(fichier);

    /* tri par sélection */
    for(iter=0;iter<taille;iter++)
    {
        imin = iter;
        /* on trouve l'indice du plus petit élément, entre les indices iter et

```

Exercice 2 – Tri à bulle

Dans cet exercice, on reprend le programme de l'exercice de *tri par sélection*, mais en utilisant un autre algorithme de tri.

On utilise le *tri à bulles* qui consiste à comparer successivement tous les éléments adjacents d'un tableau et de les échanger si le premier élément est supérieur au second.

Les détails sont donnés dans l'algorithme suivant :

Algorithme 2 : Tri à bulles

Données : T , le tableau à trier, n le nombre d'éléments de L

Sorties : T la liste triée par ordre croissant

```
1 pour  $i$  de 0 à  $n - 1$  faire
2   pour  $j$  de 1 à  $n - i - 1$  faire
3     si  $T[j - 1] > T[j]$  alors
4       échanger  $T[j - 1]$  et  $T[j]$ 
5     fin
6   fin
7 fin
```

1. Écrire le programme C correspondant (lecture des valeurs dans le fichier, tri, et sauvegarde des valeurs triées)
2. On peut constater que lorsqu'aucune permutation n'est effectuée lors d'un passage, le tableau est trié, alors les passages ultérieurs sont donc inutiles. Utiliser cette propriété pour optimiser l'algorithme précédent et écrire une nouvelle version du programme.

Solution:

```

/*=====
*
*      oO  Tri sélection  Oo
*
*=====
*
* File : tri-selection.c
* Date : 14 oct 12
* Author : Hilaire Thibault
*
*=====
*/

#include <stdio.h>
#include <stdlib.h>

#define TAILLE_MAX 100

int main()
{
    FILE* fichier;                /* descripteur de fichier */
    float T[TAILLE_MAX];          /* tableau à trier */
    float temp;
    int taille;                   /* taille du tableau */
    int i,j;

    /* ouverture du fichier */
    fichier = fopen( "valeurs.txt", "r");
    if (fichier==NULL)
    {
        printf("Problème_d'ouverture_du_fichier\n");
        return EXIT_FAILURE;
    }

    /* lecture du fichier */
    fscanf( fichier, "%d", &taille);
    for(i=0;i<taille;i++)
        fscanf( fichier, "%f", &T[i]);

    /* fermeture du fichier */
    fclose(fichier);

    /* tri à bulles */
    for(i=0;i<taille;i++)
    {
        for(j=1;j<(taille-i);j++)
            if (T[j-1]>T[j])
            {
                /* on échange T[j-1] avec T[j] */
                temp = T[j-1];
                T[j-1] = T[j];
                T[j] = temp;
            }
    }

    /* sauvegarde dans le fichier */
    fichier = fopen( "valeurs-triees.txt", "w");
    if (fichier==NULL)
    {
        printf("Problème_d'ouverture_du_fichier\n");
        return EXIT_FAILURE;
    }
    fprintf( fichier, "%d\n",taille);

```

Exercice 3 – Sudoku

Tout le monde connaît le Sudoku : il s'agit d'une grille 9×9 de chiffres allant de 1 à 9.

Le but du jeu est de remplir cette grille avec une série de chiffres tous différents, qui ne se trouvent jamais plus d'une fois sur une même ligne, dans une même colonne ou dans une même sous-grille (des carrés de 3×3).

Le but de cet exercice est de vérifier qu'une grille est correctement remplie (qu'elle vérifie donc les conditions précédemment énoncées).

Il n'est pas demandé dans cet exercice d'écrire une/des fonction(s), mais cela est possible si vous pensez y arriver.

1. Écrire un programme qui remplit un tableau 9×9 d'entiers avec les chiffres lues dans le fichier "grille.txt" du répertoire /home/sasl/shared/ei-ii/C/TP2/.

On pensera à gérer correctement les erreurs de fichier (fichier inexistant, etc.).

2. Ce programme doit ensuite afficher la grille sous la forme suivante (avec les caractères – et |) :

```
-----
| 3 | 8 | 4 | 1 | 2 | 3 | 7 | 9 | 5 |
-----
| 2 | 5 | 7 | 9 | 8 | 3 | 1 | 6 | 4 |
-----
| 9 | 1 | 6 | 7 | 5 | 4 | 3 | 2 | 8 |
-----
| 4 | 7 | 8 | 6 | 3 | 2 | 9 | 5 | 1 |
-----
| 1 | 3 | 5 | 8 | 4 | 9 | 2 | 7 | 6 |
-----
| 6 | 2 | 9 | 5 | 1 | 7 | 4 | 8 | 3 |
-----
| 8 | 4 | 3 | 2 | 9 | 5 | 6 | 1 | 7 |
-----
| 7 | 9 | 1 | 3 | 6 | 8 | 5 | 4 | 2 |
-----
| 5 | 6 | 2 | 4 | 7 | 1 | 8 | 3 | 9 |
-----
```

3. Compléter le programme pour qu'il vérifie si la grille de sudoku est correcte pour toutes les lignes
ATTENTION : cette question n'est pas évidente. Posez bien le problème, décomposez-le. Une façon de faire peut être d'utiliser un tableau (d'indice de 1 à 9) où l'on compte le nombre d'apparition de chaque chiffre de 1 à 9. À vous de jouer !

En cas de grille non correcte, vous pouvez aussi afficher un message d'erreur (du genre "condition sur ligne 2 non remplie.).

4. Compléter avec la vérification sur les colonnes.

La vérification sur les sous-tableaux est laissée en exercice pour ceux qui le souhaitent.

Solution:

```

/*=====
*
*      oO  test de grille de sudoku   Oo
*
*=====
*
* File : loto.c
* Date : 25sept 09
* Author : Hilaire Thibault
*
*=====
*/

#include <stdio.h>
#include <stdlib.h>

/* *** constantes *** */
#define DIM 3
#define TAILLE DIM*DIM

int main()
{
    int grille[TAILLE][TAILLE];      /* création du tableau de sudoku */
    FILE* fichier;                   /* pointeur de fichier */
    int i,j,k;
    int val[10];                     /* tableau pour compter le nb d'occurrence de
        chaque chiffre */

    /* ouverture du fichier */
    fichier = fopen( "grille.txt", "r");
    if (fichier==NULL)
    {
        printf("Problème_d'ouverture_du_fichier\n");
        return EXIT_FAILURE;
    }
    /* lecture du fichier */
    for(i=0;i<TAILLE;i++)
        for(j=0;j<TAILLE;j++)
            fscanf( fichier, "%d", &(grille[i][j]) );

    /* fermeture du fichier */
    fclose(fichier);

    /* affichage de la grille */
    for( i=0; i<TAILLE; i++)
    {
        /* ligne horizontale */
        for( j=0; j<TAILLE; j++)
            printf("----");
        printf("-\n");
        /* affichage de la ième ligne */
        for( j=0; j<TAILLE; j++)
            printf( "|_ %d_", grille[i][j]);
        printf("| \n");
    }
    /* dernière ligne horizontale */
    for( j=0; j<TAILLE; j++)
        printf("----");
    printf("-\n");

    /* vérification des lignes */

```